



Beyond SweepLine: Efficient MaxRS Queries over Inaccurate Location Data

Yuanjun Liu¹, Zhengcao Zhang¹, Jianfeng Qu¹, Guanfeng Liu², and An Liu^{1(✉)}

¹ School of Computer Science and Technology, Soochow University, Jiangsu, China
20214227045@stu.suda.edu.cn, {zczhang,jfqu,anliu}@suda.edu.cn

² School of Computing, Macquarie University, Sydney, Australia
guanfeng.liu@mq.edu.au

Abstract. The Maximizing Range Sum (MaxRS) query plays a pivotal role in computational geometry and location-aware applications. This query aims to optimally place a rectangle within a set of weighted objects to maximize the total weight of the objects it covers. Current algorithms for MaxRS queries, both exact and approximate, largely depend on the SweepLine algorithm. However, with its $O(n \log n)$ time complexity for n objects, this algorithm becomes inefficient for contemporary applications that handle large datasets. This paper introduces an innovative method to resolve the MaxRS query in $O(n)$ time. We first reformulate the MaxRS query as a rectangles stabbing problem. Acknowledging the presence of noise in data collected by real-world devices, we circumvent the SweepLine algorithm. Instead, we efficiently tackle the rectangles stabbing problem by optimizing space-time trade-offs. Furthermore, we expand our algorithm to address two more complex scenarios: the dynamic MaxRS (DMaxRS) query, which accounts for the varying locations of objects over time, and the MaxRS query on trajectory data (MaxRST), where each object is represented by a sequence of points, not just a singular point. Our experimental results demonstrate that our algorithms significantly outperform existing state-of-the-art methods in terms of efficiency, while maintaining reasonable memory usage and delivering satisfactory results.

Keywords: MaxRS query · Spatial dataset · Location based services

1 Introduction

The past decade has witnessed unprecedented growth in the availability of location-based services. These services necessitate the management and processing of a substantial amount of location information, drawing significant attention to technologies explored in spatial databases. Among the fundamental operations in this domain, a range of location selection queries has been extensively studied, with the MaxRS (Maximizing Range Sum) query being a notable example.

This work is supported by Natural Science Foundation of Jiangsu Province (Grant Nos. BK20211307) and the National Natural Science Foundation of China (Grant No. 52302377), and by project Funded by the Priority Academic Program Development of Jiangsu Higher Education Institutions.

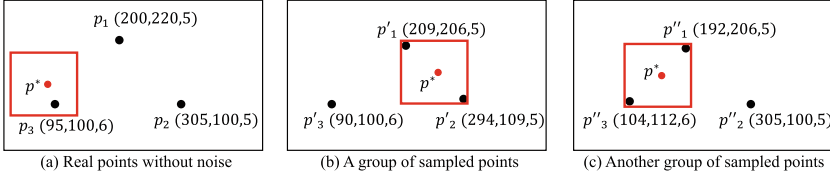


Fig. 1. Illustration of MaxRS query with real and sampled data, where each point p_i is denoted by a tuple representing its x-coordinate, y-coordinate, and weight respectively. The red square indicates the optimal placement of a rectangle that maximizes the range sum, with the point p^* indicating the optimal location for maximum coverage. (a) shows real data points without noise, marking the optimal rectangle placement for maximum coverage. (b) and (c) each display a set of sampled points with positional noise, resulting in altered optimal rectangle placements. These variations highlight the challenge of achieving exact MaxRS query results in the presence of data inaccuracies, questioning the necessity for precise algorithms like SweepLine in such contexts. (Color figure online)

Given a set of weighted points $\mathbf{P}$, the MaxRS query aims to find a point p^* to place a rectangle of size $a \times b$ to maximize the total weight of points in $\mathbf{P}$ covered by the rectangle. A classic application scenario is selecting the optimal position to serve the maximum number of clients, such as a pizza store aiming to cater to as many customers as possible within a limited delivery range [5].

The brute-force method to solve the MaxRS query is to check the range sum of infinite candidate points in space, which is impractical. The SweepLine algorithm [8, 13] can solve the query in $O(n \log n)$ time. It firstly transforms points into rectangles. Given n points, there are at most n^2 areas covered by multiple rectangles [13], so the number of candidate points is reduced to n^2 . The algorithm then employs the interval tree structure to check them exactly. Existing exact algorithms [1, 3, 5, 8, 13, 14, 23] and approximate algorithms [20, 23] for MaxRS queries predominantly rely on the SweepLine algorithm. The approximate algorithms are less complex in theory, yet consume $O(n \log n)$ time alike in practice. $O(n \log n)$ algorithms are not efficient enough for modern applications with substantial data volumes, such as optimizing signal transmission every several seconds by finding the area with maximal communication delay [1] in New York with over 8 million residents¹. The pressing need for efficiency in such high-volume data environments calls for a new foundational algorithm that overcomes the SweepLine algorithm’s constraints, enabling the practical application of MaxRS queries in today’s data-intensive scenarios.

Our work is driven by the recognition that accuracy in real-world location-based data is often compromised. For example, cellular data acquired from GSM devices can exhibit noise exceeding 100 m [16], while GPS-sampled points generally present noise around 15 m [18]. This inherent imprecision renders the pursuit of an exact solution by any algorithm futile, as demonstrated in Fig. 1. (a) represents authentic points devoid of noise, whereas (b) and (c) illustrate two distinct sets of sampled points, each affected by noise-induced displacement.

¹ <https://www.census.gov/>.

Table 1. Overview of Algorithmic Contributions, where n is the total number of points and m is the number of changed points every time.

Algorithm	Query	Data Type	Complexity	Characteristics	Section
QMax	MaxRS	Static	$O(n)$	Utilizes quantized space cells	Sect. 4
QDMax	DMaxRS	Dynamic	$O(m)$	Adapts to real-time changes	Sect. 5
QMaxT	MaxRST	Trajectory	$O(n)$	Accounts for object trajectories	Sect. 6

Consequently, the optimal range sums differ between these samples, challenging both exact and approximate algorithmic solutions. Given this context of inherent data inaccuracies, employing the SweepLine algorithm to achieve an exact solution becomes unnecessary.

Our method addresses these inaccuracies by computing an approximate solution to the MaxRS query. By dividing the space into ϵ -sized cells and representing all points within a cell by its center, we significantly simplify the problem. The rectangle is deemed to cover a cell if it contains the cell’s center, allowing us to aggregate the weights and pinpoint the cell with the maximum total weight. This approach reduces the operations to nc , where c is the count of cells within a single rectangle.

While our algorithm trades off additional space for enhanced efficiency, the memory requirements remain manageable. Take, for example, the area within Beijing’s 5th Ring Road, spanning approximately 25,000 by 25,000 m. At a cell size of 10 by 10 m, the space is partitioned into around $6.3e6$ cells, demanding a mere 24 MB of storage assuming a 32-bit representation for each cell’s weight. Even at a higher resolution of 1 by 1 m per cell, the memory footprint would be around 2.4 GB, which is well within the capacity of contemporary computing systems.

We further adapt our algorithm to tackle two additional complex scenarios. The first is the dynamic MaxRS (DMaxRS) query, which is designed to continuously monitor and update the optimal result as the locations of objects change over time. An application of this is in urban GSM systems, where there is a need to constantly observe a specific area for the greatest communication delay to fine-tune system settings and enhance signal transmission [1]. The second scenario involves the MaxRS query on trajectory data (MaxRST), which tracks object movements over time, associating each object with a trajectory composed of a sequence of points, rather than a single static point. A practical example of this would be a business seeking to identify an optimal advertising area to maximize exposure to passing potential customers [23]. Existing solutions for these scenarios are constrained by the SweepLine algorithm’s super-linear time complexity. To overcome this limitation, we introduce two novel algorithms-QDMax for DMaxRS queries and QMaxT for MaxRST queries-that operate in linear time, leveraging our quantized cell-based approach.

We summarize our contributions in Table 1. Furthermore, we have rigorously tested our algorithms against two real-world datasets to validate their effectiveness and efficiency. The subsequent section reviews previous works related to

ours, followed by Sect. 3, which defines and states the problem. Sections 4, 5, and 6 elaborate on our algorithms, and the experimental results are presented in Sect. 7. Finally, we summarize our work in Sect. 8.

2 Related Work

The existing optimal in-memory algorithm for the MaxRS query termed SweepLine [8, 13] is $O(n \log n)$ time complexity, where n is the number of points. The algorithm is firstly proposed based on searching the maximum clique of an intersection graph of rectangles [8], then an alternative and easy-to-implement version is proposed later [13]. It firstly transforms points into rectangles. Next, the rectangles are sorted according to their y -axis coordinates. Then, a line is employed to sweep from top to bottom to build the covering relations between points and calculate the range sum of each point. During sweeping, the interval tree data structure is utilized to record the activating rectangles and their positions. There are $2n$ lines to be swept, and each line consumes $O(\log n)$ to find intersection rectangles. Therefore, the SweepLine algorithm is $O(n \log n)$ time complexity and $O(n)$ space complexity.

To enhance the scalability of the SweepLine algorithm, an external-memory algorithm was proposed [5], proven to be optimal in I/O cost. Additionally, a grid-sampling-based approximate algorithm GS_SweepLine [20] provides a $(1 - \epsilon)$ -approximate answer in $O(n \log \frac{1}{\epsilon} + n \log \log n)$ time. The algorithm quantizes space into grids sized of $2a \times 2b$, where $a \times b$ is the size of query rectangle, and utilizes the SweepLine algorithm in dense grids on at most $O(\frac{1}{\epsilon^2}(\log \frac{1}{\epsilon} + \log n))$ sampled points. To ensure accuracy, the algorithm must run on 4 different groups of grids, potentially surpassing the runtime of the SweepLine algorithm. The probabilistic MaxRS query on uncertain data is explored in [12]. The index-based MaxRS query [25] pre-computes results for all possible range sizes, then searches in $O(\log n)$ time. The MaxRS query with class-based existential constraints is studied in [10]. The MaxRS query is also studied in distributed networks, such as the MaxRS query in Ad Hoc networks is explored for improved rescue operations in disaster sites [11] and the MaxRS query in distributed wireless networks for enhanced energy and communication savings [21].

Moreover, the MaxRS query has been extended to various shapes. An approximation algorithm for circular MaxRS is proposed in [5], and the polar system is employed in [9] to sweep the line annularly. Querying on diverse shapes is addressed in [7, 17], and querying on rotary rectangles is presented in [4]. The MaxRS query on the road network is solved by [2, 3, 14, 15] in $O(n \log n)$ time based on employing the SweepLine algorithm on each edge.

Several studies have focused on dynamic queries. The tree data structure is employed in [10] to handle the dynamic query on class-constrained, costing $O(mn \log n)$ time. In [6], a graph is maintained for dynamic query, and the kinetic data structure is utilized for pruning. When a point changes, it checks the connections with other points, making the algorithm $O(mn)$ time complexity. An $O(m \log n)$ algorithm [19] is proposed to find the area with the worst

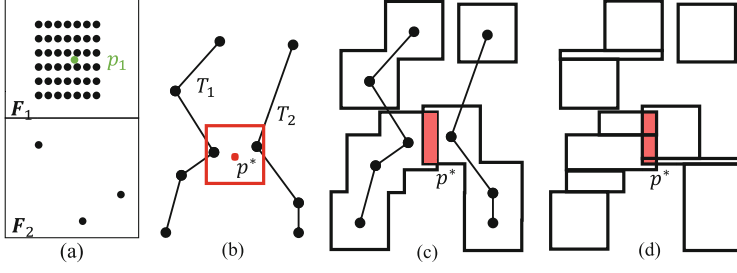


Fig. 2. (a) An example of partitioning the space into two grids F_1 and F_2 , where F_1 is densely populated with data points, and F_2 is a sparse section. p_1 is in the F_1 and will change at this time instance. (b): An example of the MaxRST query on two trajectories T_1 and T_2 , the p^* centered at the red rectangle indicates the optimal placement with the maximum range sum on trajectories. (c): The transformation of trajectories T_1 and T_2 into geometries, and the p^* in the area overlaid from both geometries is the query result. (d): Each trajectory is cut into nonoverlapping rectangles, and the p^* in the area overlaid from two rectangles is the query result. (Color figure online)

air pollution. It assumes that the pollution decreases over time according to a known equation, such a strict assumption leads to narrow applications. G2 [1] partitions the space into grids and employs the SweepLine algorithm in each changed grid. A further improvement, aG2 [1] estimates the maximal weight of each grid and reruns the SweepLine algorithm only inside grids where the weight is possibly maximized. Despite using a branch-and-bound algorithm to reduce computations, it still has a time complexity of $O(n \log n)$. Figure 2 (a) shows F_1 contains almost all points. When a few points change, e.g., p_1 appears, it needs to rerun the SweepLine algorithm in F_1 , incurring an $O(n \log n)$ time cost.

PMaxRST [23] solves the MaxRS query on trajectory data in $O(n \log n)$ time. Recall that spatial points can be transformed into rectangles, the PMaxRST algorithm converts trajectories into geometries, as shown in Fig. 2 (b) for two trajectories and (c) for corresponding geometries. It's clear that the red space in the range of two trajectories is the optimal point. Next, the PMaxRST algorithm partitions the geometry into nonoverlapping rectangles based on the SweepLine algorithm, and then uses it to find the optimal point. Figure 2 (d) shows rectangles from the same trajectories have no overlapping. Similar to the GS-SweepLine algorithm, GS.PMaxRST [23] is further proposed to find (ϵ, δ) answer in $O(n + \frac{\ln \frac{1}{1-\delta}}{ab\epsilon^2} \log \frac{\ln \frac{1}{1-\delta}}{\epsilon^2})$ time based on the grid sampling techniques. However, the algorithm is sensitive to the weight of data, and may consume more time than the PMaxRST algorithm.

3 Problem Formulation

Definition 1 (Point). $p = (x, y, w)$ is a point with the x -coordinate and y -coordinate values, e.g., the latitude and longitude, and the positive weight w .

The weight w represents the importance of objects in various scenarios, such as the communication delay that the base station wants to reduce, and the purchasing power of clients that the pizza store wants to serve.

Definition 2 (Dual Rectangle). Given the spatial point p and the size $a \times b$, $R(p)$ is an axis-aligned rectangle with the size $a \times b$ centers at p .

Given points p and q , q is in the range of p , if q is covered by the dual rectangle of p , i.e., $q \in R(p)$.

Definition 3 (MaxRS Query). Given the range size $a \times b$, n weighted spatial points P , the MaxRS query retrieves a spatial point p^* , such that the sum of the weights of points in the range of p^* is maximized, i.e.,

$$\max_{q \in R(p^*), q \in P} q.w \quad (1)$$

Definition 4 (DMaxRS Query). Given the range size $a \times b$, n weighted spatial points P , there are m points appear, disappear, move and change their weight at each time instance. The DMaxRS query continuously monitors a spatial points p^* , such that the sum of the weights of points in the range of p^* at each time instance is maximized, i.e., (1) is satisfied.

Definition 5 (Trajectory). A trajectory T is defined as a sequence of spatial points $[p_1, p_2, \dots, p_{|T|}]$ with a positive weight w .

Given a spatial point p and a trajectory T , p is in the range of trajectory T , if p is in the range of any point of T , i.e., $p \in R(q), q \in T$, as well as T is in the range of p , if any point of T is in the range of p , i.e., $q \in R(p), q \in T$.

Definition 6 (MaxRST Query). Given the range size $a \times b$ and the set of weighted trajectories T with n spatial points in total, the MaxRST query retrieves a spatial point p^* , such that the sum of the weights of trajectories in the range of p^* is maximized, i.e.,

$$\max_{q \in R(p^*), q \in T, T \in T} T.w \quad (2)$$

4 The QMax Algorithm

The brute-force method to solve the MaxRS query is to scan all possible candidate points in the space, and find the candidate point with the maximal range sum. However, there are infinite candidates in the space. Bound by real-world technology constraints, points are collected with noise, such as GSM devices have noise exceeding 100m [16], and GPS devices contain noise around 15m [18]. Therefore, we quantize the space into cells sized of $\epsilon \times \epsilon$, and treat cells as the smallest unit in the space for general purposes. One candidate point at the

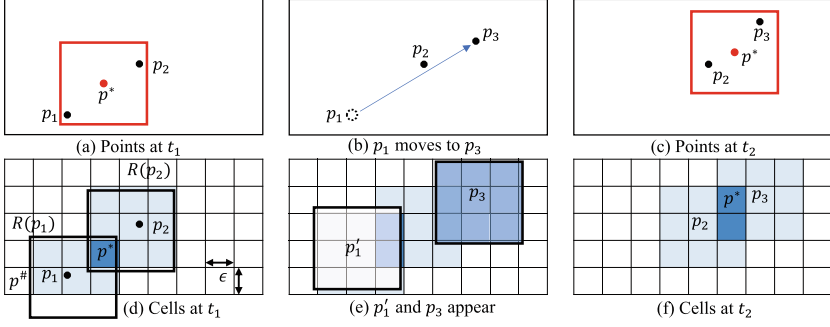


Fig. 3. (a): There are two points p_1 and p_2 at time t_1 , and p^* centered at the red rectangle that covers both two points has the maximal range sum. (b): The dynamic event at time t_1 , that is, p_1 moves to a new location p_3 . (c): The points at time t_2 after the dynamic event, the optimal result p^* changes. (d): The weights of cells of size $\epsilon \times \epsilon$ at time t_1 contributed from points p_1 and p_2 , the cell p^* lies is covered by both two dual rectangles $R(p_1)$ and $R(p_2)$, so p^* is the result at time t_1 . Meanwhile, the center of cell $p^\#$ lies isn't covered by $R(p_1)$, thus the weight of the cell is 0. (e): The dynamic event is converted to the appearance of p'_1 and p_3 , where $p'_1.w = -p_1.w$. Note that only cells in the range of p'_1 and p_3 are modified, others remain the same. (f): The weights of cells at time t_2 after the dynamic event, and the optimal result p^* alters (Color figure online).

center of a cell can represent all candidate points in the cell, and the number of candidates is reduced from infinity to the number of cells.

The brute-force method can be applied, which calculates the range sum of all cells, then finds the cell with the maximal range sum and retrieves the candidate point at the center. Without loss of generality, we assume that all points are in a rectangle space F sized of $A \times B$. After quantization, there are $\frac{AB}{\epsilon^2}$ cells on F , so the algorithm needs $\frac{AB}{\epsilon^2}$ space. Each cell goes through n points to find the points in the to calculate the range sum, thus the algorithm costs $n \frac{AB}{\epsilon^2}$ operations.

There is no need to go through all points for each candidate point, since points out of the range make no contribution to the range sum. However, finding points in the range needs at least $O(\log n)$ time based on a tree structure. Nevertheless, finding cells in the range of a point needs only $O(1)$ time, since the positions of cells are determinate. Given a point p , the cells in the range of p , i.e., covered by rectangle $R(p)$ are centered at $[R(p)]^\epsilon = \{(i\frac{\epsilon}{2}, j\frac{\epsilon}{2}) | [\frac{p.x-a/2}{\epsilon}] \leq i \leq [\frac{p.x+a/2}{\epsilon}], [\frac{p.y-b/2}{\epsilon}] \leq j \leq [\frac{p.y+b/2}{\epsilon}]\}$, where $[\cdot]$ rounds a number into an integer. Figure 3 (d) shows $R(p_1)$ covers the point p^* centered at a cell, so the cell p^* lies is in the range of p_1 . Meanwhile, $R(p_1)$ doesn't cover the point $p^\#$ centered at a cell, so the cell $p^\#$ lies isn't in the range of p_1 .

Algorithm 1 formulates the QMax algorithm. Line 1 quantizes the rectangle F into cells. The weights of cells are initialized with 0. Then for each point p , the algorithm finds the cells in the range and accumulates their weight, as lines 2–3 show. Note that when $R(p)$ is at the verge of F , e.g., the $R(p_1)$ in Fig. 3

Algorithm 1: QMax

Input: Rectangle F , precision ϵ and points P **Output:** Point p^*

```

1  $Q[F]^\epsilon \leftarrow 0$ 
2 for  $p \in P$  do
3    $Q[F \cap R(p)]^\epsilon \leftarrow Q[F \cap R(p)]^\epsilon + p.w$ 
4  $p^* \leftarrow \arg \max_{q \in Q} q.w$ 

```

(d), some cells quantized by $[R(p)]^\epsilon$ maybe not on the F , which are unnecessary since there are no candidate points out of F . So we find cells both are on F and covered by $R(p)$ by $[F \cap R(p)]^\epsilon$. Finally, line 4 finds the cell with the maximal weight and retrieves the point p^* with the maximal range sum at the center.

There are $\frac{AB}{\epsilon^2}$ cells in total, thus the space consumption is linear to AB and square to $1/\epsilon$. There are $\frac{ab}{\epsilon^2}$ cells covered by each $R(p)$, thus it costs $n\frac{ab}{\epsilon^2}$ operations to process n points. Therefore, the algorithm is $O(n)$ time complexity.

The size of the cell, i.e., ϵ is an important parameter, which balances the efficacy and efficiency. On the one hand, the bigger ϵ is, the smaller $n\frac{ab}{\epsilon^2}$ and $\frac{AB}{\epsilon^2}$ are, the less time and space consumptions the algorithm are. On the other hand, the smaller ϵ is, the more candidate points there are, the more accurate the algorithm is. The value of the ϵ should be related to the real demands. When the space and time are insufficient, the ϵ can be set bigger. On the contrary, when the accuracy is more important and space and time are abundant, the ϵ can be set smaller. The parameter ϵ is further analyzed in Sect. 7.3.

Note that cells are well-structured, allowing line 3 in the Algorithm 1 to be executed parallelly. Thus, our algorithm can be accelerated by SIMD techniques² easily, to process multiple cells simultaneously. There are many practical tools that can optimize the algorithm automatically, such as the modern GCC compiler³ and the NumPy library⁴. However, the SweepLine algorithm struggles to benefit from SIMD techniques as the sweeping and tree operations do not favor parallel execution. Moreover, for areas that aren't rectangles, one can apply the QMax algorithm on the minimum bounding box, or invoke the algorithm on multiple rectangles and retrieve the result with the maximal weight.

Both the QMax and the GS_SweepLine algorithms utilize the quantization techniques, yet their methods differ significantly. On the one hand, our algorithm sets the cell size as $\epsilon \times \epsilon$, while the GS_SweepLine algorithm sets the grid size as $2a \times 2b$. On the other hand, the GS_SweepLine algorithm invokes the SweepLine algorithm in dense grids, while our algorithm retrieves the result directly.

² https://en.wikipedia.org/wiki/Single_instruction,_multiple_data.

³ <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>.

⁴ <https://numpy.org/doc/stable/reference/simd/index.html>.

Algorithm 2: QDMax**Input:** Rectangle F and precision ϵ **Output:** Point p^* as soon as points modified

```

1  $Q[F]^\epsilon \leftarrow 0$ 
2 while points modified do
3    $P \leftarrow$  transformed appearing points
4   for  $p \in P$  do
5      $Q[F \cap R(p)]^\epsilon \leftarrow Q[F \cap R(p)]^\epsilon + p.w$ 
6    $p^* \leftarrow \arg \max_{q \in Q} q.w$ 

```

5 The QDMax Algorithm

Objects in a dynamic scenario will appear, disappear, move, and alter weight. Figure 3 depicts an instance of the DMaxRS query. (a) shows two points p_1 and p_2 at time t_1 , and p^* is the optimal result. (b) displays a dynamic event at time t_1 , that is, p_1 moves to p_3 . Lastly, (c) portrays the points at time t_2 , p^* changes.

The key to solving the dynamic query is to take full advantage of previous computations, to update changed data efficiently. Figure 2 (a) exhibits that only p_1 changes, so it's ideal to process p_1 only. However, the natures of the SweepLine algorithm make it inefficient to solve the DMaxRS query. Firstly, the SweepLine algorithm necessitates that all points are sorted. Therefore, any movement amongst the points results in disorder and a subsequent need for re-sorting. Secondly, the SweepLine algorithm doesn't maintain the cover relations of points after sweeping. Any positional changes of points will obligate the algorithm to conduct another sweep for the calculation of the range sum.

We employ quantized cells to update modified data efficiently. Our algorithm does not demand the sorting of points or the need to count the points in the range for the calculation of the range sum. When a point changes, we only need to update $\frac{ab}{\epsilon^2}$ cells around the point. By avoiding dealing with other points, we are able to update each point in $O(1)$ time and process m points in $O(m)$ time.

Figure 3 portrays an instance of the QDMax algorithm. (d) displays the weights of cells at time t_1 with two points p_1 and p_2 , p^* is the optimal result. (e) shows the event that p_1 moves to p_3 is transformed into the appearances of p'_1 and p_3 , where $p'_1.w = -p_1.w$. There are 4 kinds of dynamic events in real-life dynamic scenarios, namely, appearing, disappearing, moving, and weight-altering. One moving point can be perceived as a point disappears at the old position and a point appears at the new position. Analogously, one weight-changing point can be regarded as a point with the changed weight appears. Moreover, one disappearing point is viewed as a point appearing with opposite weight. Note that only a few cells around the changed points alter their weight, other cells remain consistent. (f) exhibits the weights of cells at time t_2 , p^* evolves dynamically.

Algorithm 2 formulates the details. In line 1, we quantize the space into cells with the initial weight of 0. Every time points change, we transform the

modified points into appearing points in line 3, and execute lines 2–3 of the QMax algorithm to update the weights of cells, then alter the p^* . The algorithm requires $m \frac{ab}{\epsilon^2}$ operations to process m points, thus is $O(m)$ time complexity. Similar to the QMax algorithm, it occupies $\frac{AB}{\epsilon^2}$ space.

6 The QMaxT Algorithm

There are two challenges in solving the MaxRST query. The first challenge is how to avoid the repetitive weight of different points from the same trajectory. As Fig. 4 (a) illustrates, two points are in the range of p' , yet only one trajectory is within the range of p' , the two points come from the same trajectory and their weights only count once. There are two points from two trajectories in the range of p^* , making it the optimal result. The second challenge is how to handle various shapes of trajectories. Trajectories have different lengths, i.e., the number of points, varying arc lengths, and irregular shapes. Consequently, algorithms that solve the MaxRS query can not tackle the MaxRST query.

We propose the QMaxT algorithm to solve the MaxRST algorithm in linear time based on quantized cells. The algorithm quantizes trajectories into cells, so the shapes of various trajectories are unified. The cells of different trajectories are isolated, and the weights of cells in the same trajectories are 0 or $T.w$, which avoids the repetitive weight. Afterward, the algorithm accumulates the cells at the same position from all trajectories and identifies the optimal result.

Figure 4 shows two trajectories T_1 and T_2 with the same weight 1. (b) illustrates T_1 is quantized into cells sized of $\epsilon \times \epsilon$, and cells in the range of T_1 have the weight 1. For each point of T_1 , we find the cells in its range and set their weight as 1. Given a cell, the cell is in the range of trajectory T_1 , if the point p at the center of the cell is in the range of T_1 , i.e., $p \in R(q), q \in T_1$. (c) shows T_2 is quantized into cells, and the cells in its range have the weight 1. Note that the cell p' lies in the range of two points, but it's in the range of one trajectory. So the cell has the weight 1 instead of 2. (d) portrays the cells from all trajectories are accumulated, then the p^* with the maximal range sum is retrieved.

In practice, we employ 2 groups of cells circularly instead of generating cells for each trajectory. Let $\mathbf{Q}$ be the final cells accumulated from all trajectories, and $\mathbf{M}$ be the cells quantized by each trajectory. For each trajectory T , we set the weights of all cells of $\mathbf{M}$ be 0, and set the weights of cells in its range be $T.w$, then add the weights of $\mathbf{M}$ to $\mathbf{Q}$. By recycling the cells $\mathbf{M}$, the space consumption of the algorithm remains constant rather than linear to the number of trajectories, with the time consumption staying the same.

The detail is formulated in the Algorithm 3. In line 1, the rectangle space F is quantized into 2 groups of cells $\mathbf{Q}$ and $\mathbf{M}$ sized of $\epsilon \times \epsilon$ with initial weight 0. In lines 3–6, the algorithm quantizes each trajectory T into cells and then overlays them together. In line 3, the algorithm resets cells $\mathbf{M}$ with the initial weight 0. In line 5, cells $\mathbf{M}$ covered by trajectory T , i.e., the dual rectangles of points in T , have the weight of $T.w$. The maximum weight of cells $\mathbf{M}$ is $T.w$, which avoids the duplicate weight of points from the same trajectory. In line 6,

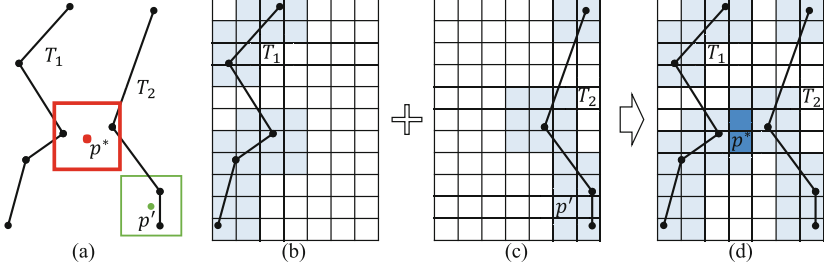


Fig. 4. (a) There are two trajectories T_1, T_2 with the same weight 1. p^* centered at the red rectangle covered two points from two trajectories is the optimal result, and p' centered at the green rectangle covered two points from the one trajectory isn't optimal. (b): The space encompassing T_1 is quantized into cells, and the cells in the range of T_1 , i.e., covered by the dual rectangle of any point of T_1 , have the weight 1. (c): The space containing T_2 is quantized into cells, and the cells in the range of T_2 have the weight 1. Note that the cell p' lies has the weight 1 instead of 2. (d): We overlay cells from all trajectories, and find the p^* with the maximal range sum centered at the cell with the maximal weight as the optimal result (Color figure online).

Algorithm 3: QMaxT

Input: Rectangle F , precision ϵ and trajectories T

Output: Point p^*

```

1  $Q[F]^\epsilon \leftarrow 0, M[F]^\epsilon \leftarrow 0$ 
2 for  $T \in T$  do
3    $M[F]^\epsilon \leftarrow 0$ 
4   for  $p \in T$  do
5      $M[F \cap R(p)]^\epsilon \leftarrow T.w$ 
6    $Q \leftarrow Q + M$ 
7  $p^* \leftarrow \arg \max_{q \in Q} q.w$ 
  
```

the algorithm adds the weights of M to Q , while Q is accumulated from all trajectories. Finally, the p^* with the maximal range sum is retrieved in line 7.

There are $\frac{AB}{\epsilon^2}$ cells in Q and M , so the total space consumption is $2\frac{AB}{\epsilon^2}$. The time consumption is $n\frac{ab}{\epsilon^2}$, where n is the total number of points from all trajectories. Therefore, the QMaxT algorithm is $O(n)$ time complexity.

Both the QMaxT and GS.PMaxRST algorithms utilize the quantization techniques, yet the sizes of the quantized space differ and our algorithm retrieves the result directly instead of invoking the SweepLine algorithm.

7 Experiment

In this section, we execute sufficient experiments to verify the efficiency of our algorithms. Section 7.1 gives the details of the experimental settings. In Sect. 7.2,

we evaluate the performances of different algorithms in different queries. Finally, we analyze the effect of the parameter of our algorithms in Sect. 7.3.

7.1 Experimental Settings

Dataset. Following [1, 23], we conduct experiments on two trajectory datasets collected in Beijing, China. T-drive [22] contains 8e4 trajectories and 1.3e7 points from 1e4 taxis from Feb. 2 to Feb. 8, 2008, and GeoLife [24] contains 1.6e4 trajectories and 1.6e7 points from 182 users from Apr. 2007 to Aug. 2012 after preprocessing. Table 2 shows the details of the two datasets. We select data in the main urban areas, i.e., the 5th Ring Road of Beijing⁵ within longitude $\in [116.25^\circ, 116.5^\circ]$ and latitude $\in [39.8^\circ, 40.05^\circ]$. So the points are in the rectangle area F sized of $A \times B$, where $A = B = 0.25^\circ$, about 25,000 m.

For the MaxRS and DMaxRS queries, we select n points randomly and generate the weights uniformly in the range $[0, 1]$. For the MaxRST query, we select trajectories incrementally until the number of points reaches n , then generate the weights of trajectories uniformly in the range $[0, 1]$. For the DMaxRS query, we select $m \leq n$ points changes at each time instance, indicating the appearing, disappearing, moving, and weight changing of objects.

Table 2. Datasets

Dataset	#Trajectories	#Points
T-drive [22]	80,359	13,918,199
GeoLife [24]	16,222	16,285,538

Table 3. Parameters

Parameter	Values
n	$10^2, 10^3, 10^4, \mathbf{10^5}$
$m(\times n)$	0.01%, 0.1%, 1% , 10%, 100%
a, b, d (meter)	100, 200, 300 , 400, 500
ϵ, ε (meter)	10, 15 , 20, 30, 40, 50

Parameters. Table 3 shows the parameter settings. We set the number of total points, i.e., the n , from 10^2 to 10^5 , and set $n = 10^5$ by default. We believe that the number of changing points at each time instance, i.e., the m , should be related to the number of total points, so we set it from 0.01% n to 100% n , and set $m = 1\%n$ by default. We set the range size, i.e., the a and b , from 100 m to 500 m, and set them to 300 m by default. We set the range of noise, i.e., the ε , from 10 to 50 m, and set it to 15 m by default, since points collected by GPS device have the noise about 15 m [18]. We set the size of cells of our algorithms, i.e., the ϵ , from 10 to 50 m, and set $\epsilon = \varepsilon$ by default.

Evaluation Metrics. We compare the runtimes and space consumptions to evaluate the efficiency, and employ the Mean Absolute Error (MAE) to evaluate the effectiveness. Given data with the set of points $\mathbf{P}$, we run the ground-truth methods gt , i.e., the SweepLine algorithm for the MaxRS query and the

⁵ <https://www.openstreetmap.org/>.

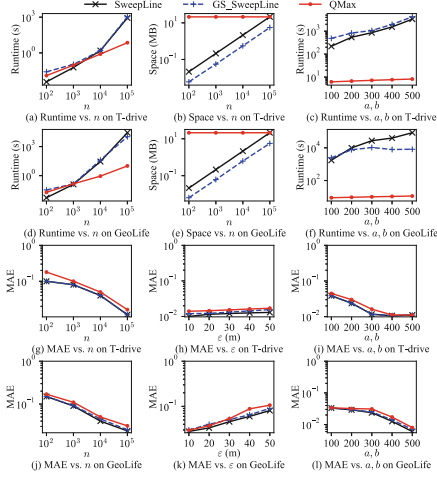


Fig. 5. The performance of MaxRS query.

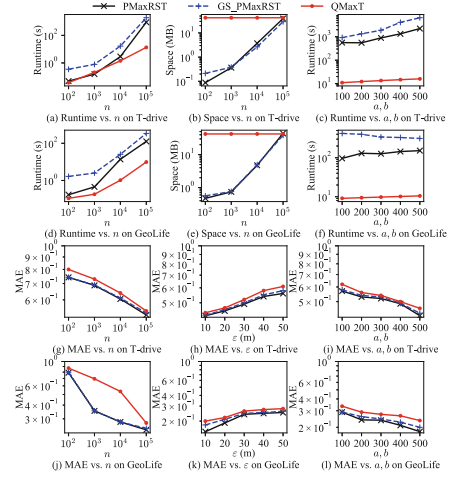


Fig. 6. The performance of MaxRST query.

PMaxRST query for the MaxRST query, on $\mathbf{P}$ to get the optimal maximal range sum w_{gt} . Next, we add random noise in the range of ϵ on $\mathbf{P}$ to simulate the sampling of devices, which yields $\mathbf{P}'$, then run each method f on $\mathbf{P}'$ to get the maximal range sum w'_f under noise. Finally, we calculate the $MAE_f = \frac{|w_{gt} - w'_f|}{w_{gt}}$.

Note that the MAE of ground-truth methods will not be zero, since they can't retrieve the optimal result under noise, i.e., $w'_{gt} \neq w_{gt}$. Therefore, an effective algorithm should have a similar MAE with the ground-truth method, and an efficient algorithm should have a shorter runtime and a lower space consumption.

Compared Algorithms. We compare the following algorithms to solve various of MaxRS queries. All of them are implemented in Python. For the MaxRS query, we compare the SweepLine [13], the GS_SweepLine [20], and the QMax algorithms. For the DMaxRS query, we compare the aG2 [1] and the QDMax algorithms. For the MaxRST query, we compare the PMaxRST [23], the GS_PMaxRST [23], and the QMaxT algorithms. Following [1, 20, 23], we set the ϵ of GS_SweepLine algorithm as 1%, set the grid size of the aG2 algorithm as $\frac{A}{1000} \times \frac{B}{1000}$, and set the ϵ and δ of the PMaxRST algorithm as 0.1 and 0.95.

The following experiments show that, our algorithms are much faster than others, the spaces of our algorithms are acceptable, and the results of our algorithms are similar to exact algorithms.

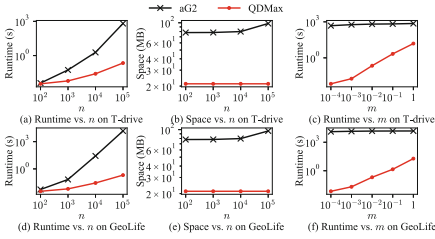


Fig. 7. The performance of DMaxRS query.

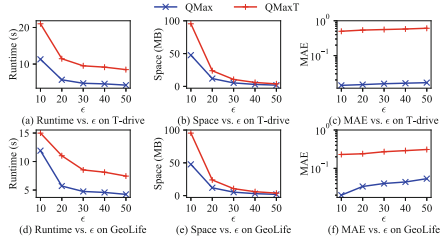


Fig. 8. The effect of parameter ϵ .

7.2 Performances on Various Queries

Performances on MaxRS Query. We compare the time and space consumptions in Fig. 5 (a) to (f), and the MAEs in Fig. 5 (g) to (l).

Figure 5 (a) and (d) show the runtimes of algorithms increase with the number of points n , and (c) and (f) show the runtimes of algorithms increase with the range size a, b . Note that the runtimes and the increments of runtimes of the QMax algorithm are slower than others. Specifically, when $n = 10^5$ and $a = b = 500$ meters, the QMax algorithm is about 1,000 times faster than others. The runtimes of the approximate GS.SweepLine algorithm are sometimes higher than the SweepLine algorithm, since the GS.SweepLine algorithm must run 4 times on different groups of grids to ensure accuracy.

Figure 5 (b) and (e) show the space consumption of the QMax algorithm is constant, but the consumptions of other algorithms increase with n , and will exceed the QMax algorithm when $n > 10^5$. Note that the space consumption of the QMax algorithm is less than 100 MB under the default settings, which is acceptable for modern computers.

Figure 5 (g) to (l) show the MAEs of algorithms on various conditions. On the whole, the MAEs of the QMax algorithm are similar to others, not only the values, but also the tendencies. When n and a, b increase, the error caused by individual points is no more attractive than the maximal range sum, thus the MAEs decrease. When ε , i.e., the noise of data increases, the errors increase.

Performances on DMaxRS Query. We compare the time and space consumptions in Fig. 7, and omit the MAEs limited by space, and the MAEs of the MaxRS query and the DMaxRS query are similar.

Figure 7 (a) and (d) show the runtimes of algorithms increase with the number of points n . The runtimes and the increments of runtimes of the QDMax algorithms are lower than the aG2 algorithm. Figure 7 (b) and (e) show the space consumption of the QDMax algorithm is constant, but the consumptions of the aG2 algorithms increase with n , and are higher than the QDMax algorithm. Figure 7 (c) and (f) show the runtimes of algorithms increase with the number of changed points at each time instance m . The aG2 algorithm consumes lots of time even if m is small, i.e., a few points changed. While the QDMax algorithm

only needs to update cells around changed points, its time consumption is linear m .

Performances on MaxRST Query. We compare the time and space consumptions in Fig. 6 (a) to (f), and the MAEs in Fig. 6 (g) to (l).

Figure 6 (a) and (d) show the runtimes of algorithms increase with the number of points n , and (c) and (f) show the runtimes of algorithms increase with the range size a, b . The runtimes of the QMaxT algorithm are smaller than others, and the increments of runtimes of the QMaxT algorithm are lower than others. Similar to the performances of the GS_SweepLine algorithm, the runtimes of GS_PMaxRST algorithm are sometimes even higher than the PMaxRST algorithm, since it must run 4 times on different groups of grids to ensure accuracy.

Figure 6 (b) and (e) show the space consumption of the QMaxT algorithm is constant, and less than 100 MB, which is acceptable for modern computers. However, the space consumptions of other algorithms increase with n , and will exceed the QMaxT algorithm when $n > 10^5$.

Figure 6 (g) to (l) show that the MAEs decrease when n and a, d increase, and increase with ε . The MAEs of the QMaxT algorithm are similar to the MAEs of others, not only the values, but also the tendencies, in a nutshell. The MAEs decrease when n and a, d increase, and increase when ε , i.e., the noise of data increases. Note that the MAEs of the MaxRST query are higher than the MaxRS query in general, since there are multiple points in each trajectory, which reduces the maximal range sum of the MaxRST query. Take the T-drive dataset for example, given the limitation $n = 10^5$, there are about 360 trajectories with a maximal range sum of about 28 for the MaxRST query, but 10^5 individual points with a maximal range sum of more than 400 for the MaxRS query. The bigger the maximal range sum is, the less attractive the error caused by individuals is, the lower the MAE is, and vice versa.

7.3 Parameter Analysis

Figure 8 (a) and (d) show the runtimes of algorithms decay when ϵ increases. The bigger ϵ is, the fewer cells the algorithms are, the less runtime algorithms cost. Note that the runtime of the QMaxT algorithm on the T-drive dataset is bigger than the GeoLife dataset. Table 2 shows that the average length of trajectories of GeoLife is bigger than T-drive, there are fewer trajectories of GeoLife under the limitation n , so lines 3 and 6 of the Algorithm 3 spend fewer time.

Figure 8 (b) and (e) show the space consumptions of algorithms decrease when ϵ increases, since there are fewer cells with a bigger ε . The space consumptions of our algorithms are linear to $A \times B$ and square to $1/\epsilon$, but constant to n .

Figure 8 (c) and (f) show the MAEs of algorithms increase when ϵ increases. The bigger ϵ is, the fewer cells are, the fewer candidate points are, and the bigger the MAE is. As mentioned above, the MAEs of the QMaxT algorithm are higher than the QMax algorithm, since the maximal range sum of the MaxRST query is less and more susceptible to the influence of noise.

There are two observations that can help with choosing the ϵ . Firstly, the smaller the ϵ is, the more space and time our algorithms consume, and the less MAE our algorithms yield. Therefore, the ϵ can be set as small as possible under the limitation of expected time and memory capacity. Secondly, the MAE decays when ϵ reduces, especially when $\epsilon \leq \varepsilon$, setting $\epsilon \leq \varepsilon$ can generally yield a result with a low MAE. In short, we suggest that the ϵ can be set as ε for general applications, and be set as small as possible for higher accuracy.

8 Conclusion

This paper investigated the MaxRS query, dynamic MaxRS query, and the MaxRS query on trajectory data. Existing optimal algorithms for various MaxRS queries are founded on the SweepLine algorithm, so all of them are $O(n \log n)$ time complexity. The catalyst for our study stemmed from the fact that most real-world location-based data inherently carry noise. We quantized the space into cells and used one candidate point at the center to represent infinite candidate points in one cell. Based on quantized cells, we solved the MaxRS and MaxRST queries in $O(n)$ time, and solved the DMaxRS query in $O(m)$ time. We executed sufficient experiments on two real datasets. On the one hand, our algorithms significantly outperformed existing methods with acceptable space consumption. On the other hand, the results of our algorithms were in accordance with others.

References

1. Amagata, D., Hara, T.: Monitoring MaxRS in spatial data streams. In: EDBT, pp. 317–328 (2016)
2. Chen, J., Zhang, K.-Q., Ren, T., Wu, Z.-Q., Gao, H.: GAM: a GPU-Accelerated Algorithm for MaxRS queries in road networks. *J. Comput. Sci. Technol.* **37**(5), 1005–1025 (2022). <https://doi.org/10.1007/s11390-022-2330-3>
3. Chen, Z., Yuan, Q., Liu, W.: Monitoring best region in spatial data streams in road networks. *Data Knowl. Eng.* **120**, 100–118 (2019)
4. Chen, Z., Liu, Y., Wong, R.C.W., Xiong, J., Cheng, X., Chen, P.: Rotating MaxRS queries. *Inf. Sci.* **305**, 110–129 (2015)
5. Choi, D.W., Chung, C.W., Tao, Y.: A scalable algorithm for maximizing range sum in spatial databases. *Proc. VLDB Endow.* **5**(11), 1088–1099 (2012)
6. Hussain, M.M., Islam, K.A., Trajcevski, G., Ali, M.E.: Towards efficient maintenance of continuous maxrs query for trajectories. In: EDBT, pp. 402–413 (2017)
7. Hussain, M.M.u., Trajcevski, G.: Maximizing area-range sum for spatial shapes (MaxRS3). In: SSDBM, pp. 1–4 (2018)
8. Imai, H., Asano, T.: Finding the connected components and a maximal clique of an intersection graph of rectangles in the plane. *J. Algorithms* **4**(4), 310–323 (1983)
9. Liu, Q., Zheng, L., Lian, X., Chen, L.: Efficiently discovering regions of interest with user-defined score function. In: DASFAA, pp. 591–608 (2021)
10. Mostafiz, M.I., Mahmud, S.M.F., Hussain, M.M.u., Ali, M.E., Trajcevski, G.: Class-based conditional MaxRS query in spatial data streams. In: SSDBM, 1–12 (2017)

11. Nakayama, Y., Amagata, D., Hara, T.: An efficient method for identifying MaxRS location in mobile ad hoc networks. In: DEXA, pp. 37–51 (2016)
12. Nakayama, Y., Amagata, D., Hara, T.: Probabilistic MaxRS queries on uncertain data. In: DEXA, pp. 111–119 (2017)
13. Nandy, S., Bhattacharya, B.: A unified algorithm for finding maximal and minimum object enclosing rectangles and cuboids. *Comput. Math. Appl.* **29**(8), 45–61 (1995)
14. Phan, T.K., Jung, H., Kim, U.M.: An efficient algorithm for maximizing range sum queries in a road network. *Sci. World J.* **2014**, 541602 (2014)
15. Phan, T., Jung, H., Youn, H.Y., Kim, U.: Efficient evaluation of maximizing range sum queries in a road network. *IEICE Trans. Inf. Syst.* **99-D**(5), 1326–1336 (2016)
16. Shi, W., Xu, J., Fang, J., Chao, P., Liu, A., Zhou, X.: LHMM: a learning enhanced hmm model for cellular trajectory map matching. In: ICDE, pp. 2429–2442 (2023)
17. Skoutas, D., Sacharidis, D., Patroumpas, K.: Discovering mixture-based best regions of arbitrary shapes. In: SIGSPATIAL, pp. 468–479 (2021)
18. Ta, N., Li, G., Xie, Y., Li, C., Hao, S., Feng, J.: Signature-based trajectory similarity join. *IEEE Trans. Knowl. Data Eng.* **29**(4), 870–883 (2017)
19. Tahmasbi, A., Trajcevski, G.: Maximal range-sum for dynamically occurring objects with decaying weights. In: ADBIS, vol. 13389, pp. 238–252 (2022)
20. Tao, Y., Hu, X., Choi, D.W., Chung, C.W.: Approximate MaxRS in spatial databases. *Proc. VLDB Endow.* **6**(13), 1546–1557 (2013)
21. Wonge-ammat, P., Mas-ud Hussain, M., Trajcevski, G., Avci, B., Khokhar, A.: Distributed in-network processing of k-MaxRS in wireless sensor networks. In: SENSORNETS, pp. 108–117 (2017)
22. Yuan, J., et al.: T-drive: driving directions based on taxi trajectories. In: ACM-GIS 2010, pp. 99–108 (2010)
23. Zhang, K., Gao, H., Han, X., Chen, J., Li, J.: Maximizing range sum in trajectory data. In: ICDE, pp. 755–766 (2022)
24. Zheng, Y., Xie, X., Ma, W.: GeoLife: a collaborative social networking service among user, location and trajectory. *IEEE Data Eng. Bull.* **33**(2), 32–39 (2010)
25. Zhou, X., Wang, W., Xu, J.: General purpose index-based method for efficient MaxRS query. In: DEXA, pp. 20–36 (2016)